

Othello[®]



ITE TO LEARN... A LIFETIME TO MASTER™

PROYECTO 3: JUEGO COMPETITIVO DE OTHELLO UTILIZANDO ALPHA- BETA PRUNING, MONTE CARLO TREE SEARCH Y HEURÍSTICAS ADAPTATIVAS

María José Ramírez | Daniel Machic | Marcos Diaz

»»» INTRODUCCIÓN «««

El juego **Othello (Reversi)** constituye un problema clásico de Inteligencia Artificial debido a la necesidad de tomar decisiones estratégicas dentro de un amplio espacio de búsqueda. Para abordar este problema, se implementaron y compararon dos enfoques de toma de decisiones: **Alpha-Beta Pruning** y **Monte Carlo Tree Search (MCTS)**, acompañados de una función heurística adaptativa para evaluar posiciones del tablero.

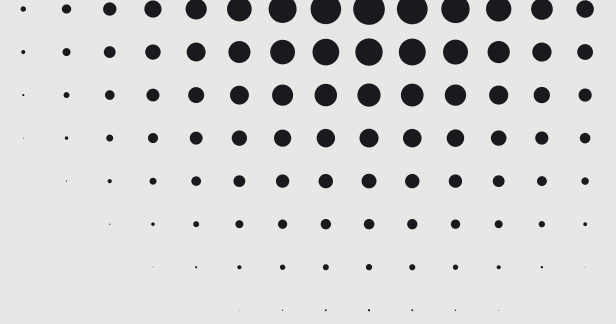
El objetivo del proyecto fue analizar el desempeño de ambos algoritmos mediante partidas automáticas, evaluando métricas como victorias obtenidas, tiempo de búsqueda y cantidad de estados explorados, con el fin de identificar sus ventajas y limitaciones en un entorno competitivo.

MARCO TEÓRICO

Othello (Reversi)

El Othello es un juego de estrategia para dos jugadores en un tablero de 8×8 . El objetivo es finalizar la partida con una mayor cantidad de fichas que el oponente mediante la captura y volteo de fichas rivales.





MINIMAX

El algoritmo Minimax es una técnica de búsqueda adversarial utilizada en juegos de suma cero como Othello. Explora un árbol de decisiones alternando entre movimientos propios y del oponente, asumiendo que ambos juegan de forma óptima.

Su objetivo es seleccionar la jugada que maximiza la ventaja del jugador y minimiza la del adversario. Aunque produce decisiones de alta calidad, el número de estados evaluados aumenta rápidamente conforme crece la profundidad de búsqueda.

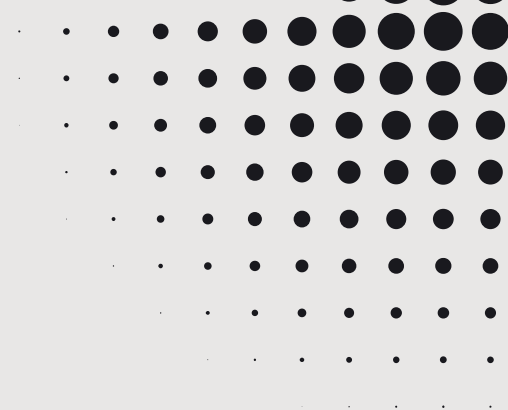
ÁRBOL DE BUSQUEDA MINIMAX

Representa las posibles secuencias de movimientos que pueden generarse durante una partida de Othello. Cada nodo corresponde a un estado del tablero y cada rama representa una acción válida que puede realizar un jugador.

- Los nodos MAX buscan maximizar la ventaja del jugador.
- Los nodos MIN representan al oponente y buscan minimizar dicha ventaja.
- Los estados finales o estados límite se evalúan mediante una función heurística.

El algoritmo selecciona la jugada que conduce al mejor resultado posible considerando la respuesta óptima del adversario.

ALPHA-BETA PRUNING



Alpha-Beta es una optimización del algoritmo Minimax que reduce la cantidad de estados evaluados durante la búsqueda. Su funcionamiento consiste en descartar ramas del árbol que no pueden influir en la decisión final.

- Alpha α : representa la mejor valoración encontrada para el jugador MAX.
- Beta β : representa la mejor valoración encontrada para el jugador MIN.
- Cuando se cumple $\alpha \geq \beta$, la rama restante puede descartarse mediante poda.
- Permite obtener la misma decisión que Minimax explorando considerablemente menos nodos.

ÁRBOL DE BUSQUEDA ALPHA-BETA

Alpha-Beta utiliza el mismo árbol de búsqueda que Minimax, donde cada nodo representa un estado del tablero y cada rama corresponde a un movimiento posible. La diferencia principal es que incorpora un mecanismo de poda para evitar explorar estados que no afectan la decisión final.

- Los nodos MAX buscan maximizar la ventaja del jugador.
- Los nodos MIN representan al oponente y buscan minimizar dicha ventaja.
- Los valores Alpha α y Beta β establecen límites para identificar ramas irrelevantes.
- Cuando una rama no puede producir una mejor decisión que las ya encontradas, se elimina mediante poda.
- Permite obtener el mismo resultado que Minimax explorando significativamente menos estados.

MONTE CARLO TREE SEARCH (MCTS)

Monte Carlo Tree Search es un algoritmo de búsqueda basado en simulaciones que estima la calidad de los movimientos mediante la exploración progresiva de un árbol de juego. Cada nodo representa un estado del tablero y se evalúa a partir de los resultados obtenidos en múltiples partidas simuladas.

- Construye el árbol de búsqueda de forma incremental a medida que realiza simulaciones.
- Utiliza la estrategia UCT (Upper Confidence Bound applied to Trees) para equilibrar exploración y explotación.
- Evalúa los movimientos según la cantidad de victorias obtenidas durante las simulaciones.
- Actualiza las estadísticas del árbol mediante un proceso de retropropagación.
- Permite tomar decisiones efectivas sin necesidad de explorar exhaustivamente todos los estados posibles.

ÁRBOL DE BUSQUEDA MCTS

El árbol de búsqueda en Monte Carlo Tree Search se construye de manera incremental mediante simulaciones sucesivas. Cada nodo representa un estado del tablero y almacena estadísticas sobre visitas y victorias obtenidas durante las simulaciones.

- La búsqueda se divide en cuatro etapas: selección, expansión, simulación y retropropagación.
- Los nodos más prometedores se seleccionan utilizando la estrategia UCT.
- Nuevos estados se agregan al árbol conforme avanza la exploración.
- Las simulaciones permiten estimar la calidad de los movimientos sin explorar exhaustivamente todas las posibilidades.
- Las estadísticas acumuladas guían la selección del movimiento con mayor probabilidad de éxito.

FUNCIONES HEURÍSTICAS

- Las funciones heurísticas permiten evaluar estados no terminales del juego sin explorar completamente el árbol de búsqueda. En este proyecto se consideraron factores como movilidad, control de esquinas, control de bordes y diferencia de fichas.

BÚSQUEDA ADVERSARIAL

- La búsqueda adversarial modela problemas donde dos agentes compiten por objetivos opuestos. Estos métodos son ampliamente utilizados en juegos de estrategia para anticipar movimientos futuros y seleccionar acciones óptimas.

PESOS DE LAS FUNCIONES HEURÍSTICAS

Factor	Apertura	Medio Juego	Final
Movilidad	5	4	2
Esquinas	30	40	40
Bordes	3	5	3
Posiciones X	10	8	4
Posiciones C	8	6	2
Diferencia de fichas	1	2	10

FASES DE LA HEURÍSTICA

Fase (segun cantidad de movimientos)	Criterio Principal
Apertura (<20)	Movilidad
Medio Juego (20-49)	Esquinas y bordes
Final (> o igual a 50)	Diferencia de fichas

ESTADO DEL JUEGO Y ESPACIO DE BÚSQUEDA

- Representa todas las configuraciones posibles del tablero de Othello y los movimientos legales disponibles para cada jugador. Cada estado puede generar múltiples estados sucesores, formando el árbol de búsqueda utilizado por los algoritmos de decisión.
- Cada estado corresponde a una configuración específica del tablero 8×8.
- Las acciones representan movimientos válidos para el jugador en turno.
- Los estados sucesores se generan aplicando las reglas de captura y volteo de fichas.
- El espacio de búsqueda está compuesto por todas las secuencias posibles de movimientos durante la partida.

RESTRICCIONES DEL JUEGO

- Solo pueden realizarse movimientos que capturen al menos una ficha rival.
- Las fichas capturadas cambian automáticamente al color del jugador.
- Si un jugador no posee movimientos válidos, pierde su turno.
- La partida finaliza cuando el tablero está lleno o ninguno de los jugadores puede realizar movimientos.



:: :: :: :: :: :: :: METODOLOGÍA :: :: :: :: :: :: ::

- **Implementación del juego:** Se desarrolló un motor completo de Othello capaz de gestionar el estado del tablero, validar movimientos, aplicar las reglas de captura y determinar condiciones terminales de la partida.
- **Implementación de agentes:** Se desarrollaron agentes basados en Minimax, Alpha-Beta Pruning y Monte Carlo Tree Search (MCTS). Además, se incorporó una función heurística adaptativa para evaluar posiciones del tablero en distintas fases del juego.
- **Evaluación de desempeño:** Los algoritmos fueron comparados mediante partidas automáticas, registrando métricas como:
 - Victorias obtenidas.
 - Tiempo de búsqueda.
 - Nodos explorados.
 - Factor de ramificación efectivo.
- **Visualización de resultados:** Se generaron gráficas comparativas para analizar el desempeño de los agentes en términos de eficiencia computacional y calidad de las decisiones tomadas.

: : : : : :

LIBRERIAS

: : : : : :

- **Pygame:** Librería para el desarrollo de aplicaciones gráficas y videojuegos. Se utilizó para implementar la interfaz gráfica del juego, visualizar el tablero y permitir la interacción entre jugadores y agentes inteligentes.
- **Random:** Generación de números y elecciones aleatorias. Lo utilizamos para crear la población inicial, realizar cruces, mutaciones y selección en el algoritmo genético.
- **Time:** Herramienta para medir tiempo de ejecución. Registrar y comparar tiempos de cómputo de GA y MTZ.
- **Pyplot:** Para la creación de gráficas.
- **Math:** Diversas funciones matemáticas

EXPLICACIÓN

1. Función Heurística:

- a. Evalúa estados no terminales del juego.
- b. Considera movilidad, control de esquinas, control de bordes y diferencia de fichas.
- c. Utiliza pesos distintos según la fase de la partida.

2. Minimax:

- a. Explora el árbol de búsqueda alternando entre los jugadores MAX y MIN.
- b. Selecciona la jugada que maximiza la ventaja del jugador considerando una respuesta óptima del oponente.

3. Poda Alpha-Beta:

- a. Descarta ramas que no pueden afectar la decisión final.
- b. Reduce significativamente la cantidad de estados evaluados.
- c. Permite alcanzar mayores profundidades de búsqueda.

4. Selección UCT (MCTS):

- a. Balancea exploración y explotación durante la búsqueda.
- b. Prioriza nodos prometedores sin dejar de explorar nuevas alternativas.

5. Simulación:

- a. Realiza partidas automáticas desde un estado determinado.
- b. Los resultados obtenidos permiten estimar la calidad de cada movimiento.

6. Retropropagación:

- a. Actualiza estadísticas de visitas y victorias en los nodos recorridos.
- b. La información acumulada se utiliza para mejorar decisiones futuras.

VISUALIZACIÓN GRÁFICA

Othello AI

OTHELLO

Human vs AlphaBeta

Turn: Black

Black: 2

White: 2

Nodes: 0

Time: 0.000s

Eval: 0.0

1:H-H 2:H-AB

3:H-MCTS 4:AB-MCT

R: Reset

Black	White	Winner
49	15	AlphaBeta
52	12	AlphaBeta
30	34	MCTS

Resultados: Torneos



Torneo Alpha-Beta vs MCTS

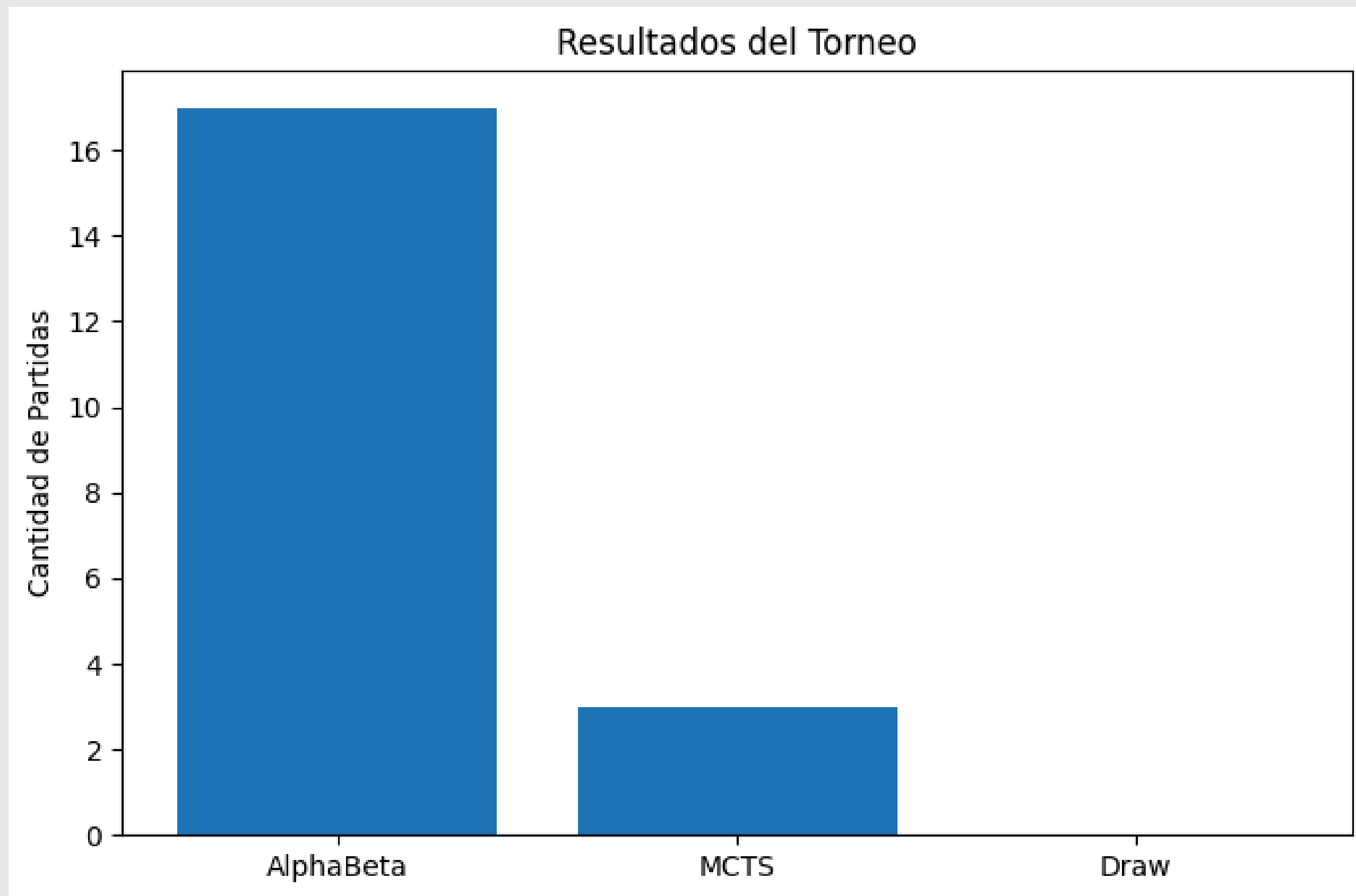


Se realizaron 20 partidas automáticas entre los agentes Alpha-Beta y Monte Carlo Tree Search (MCTS).

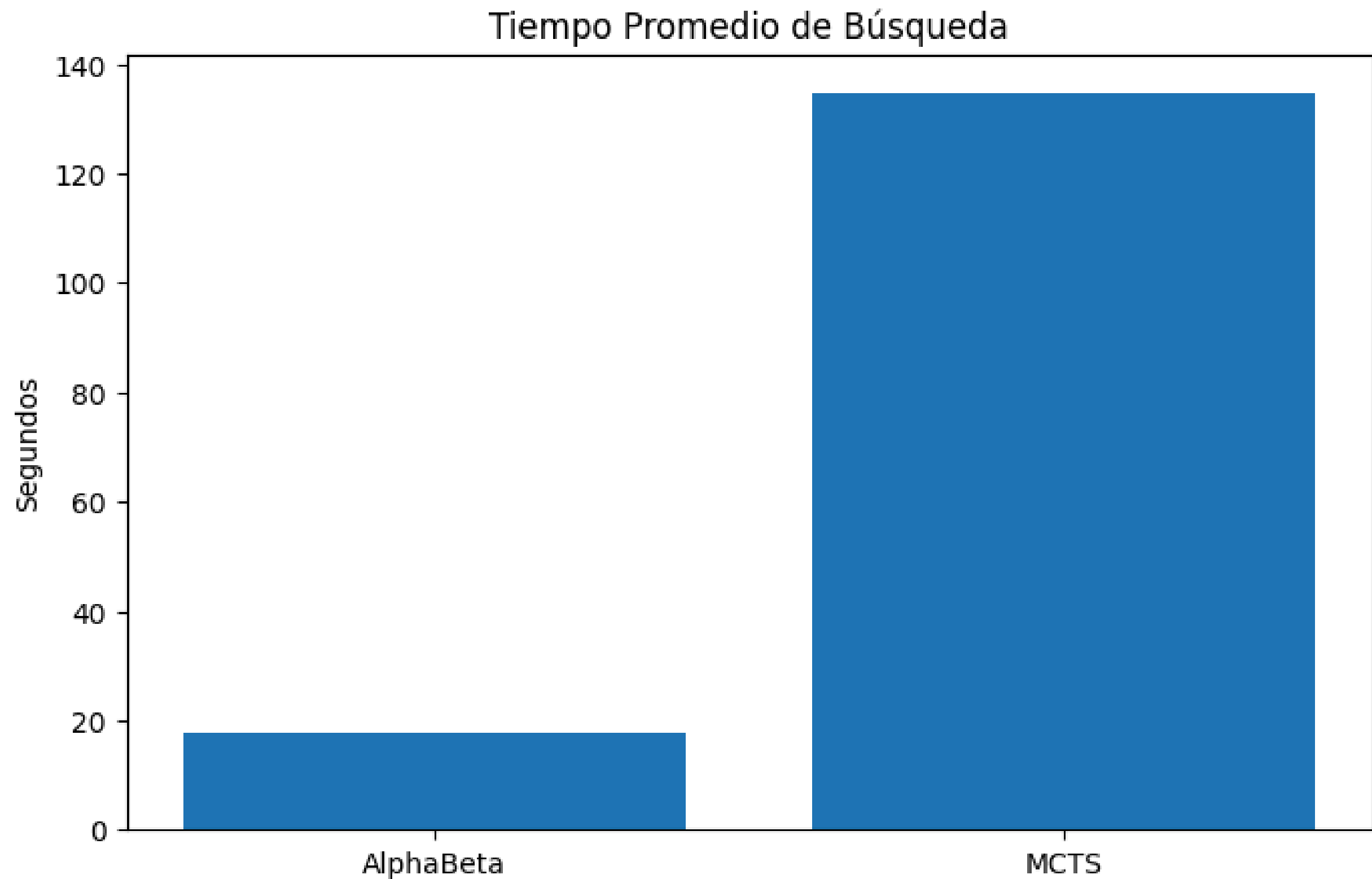
- Alpha-Beta obtuvo 17 victorias (85%).
- MCTS obtuvo 3 victorias (15%).
- No se registraron empates.

Resultado	Cantidad
Alpha-Beta	17
MCTS	3
Empates	0

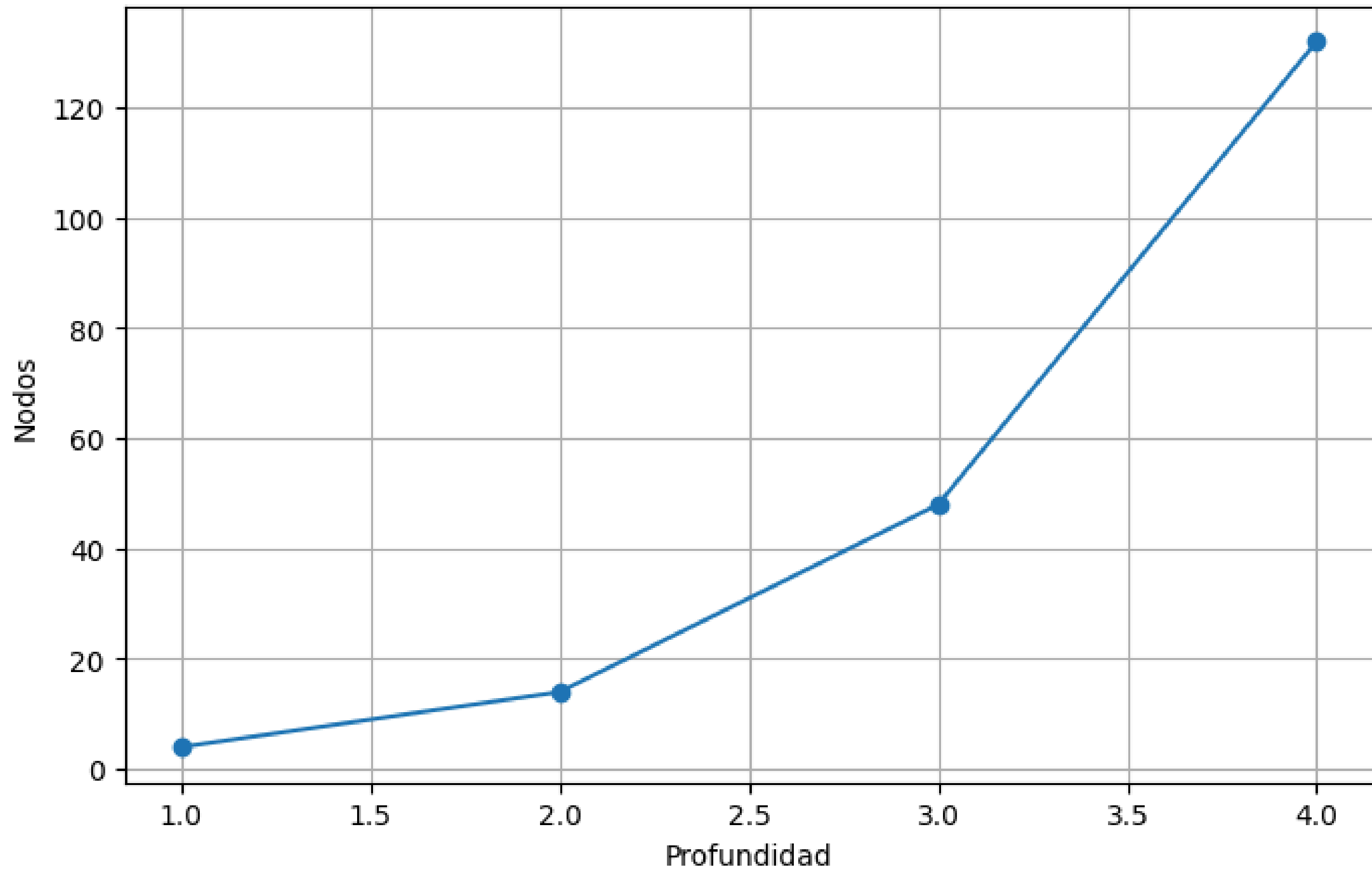
Resultados Torneo



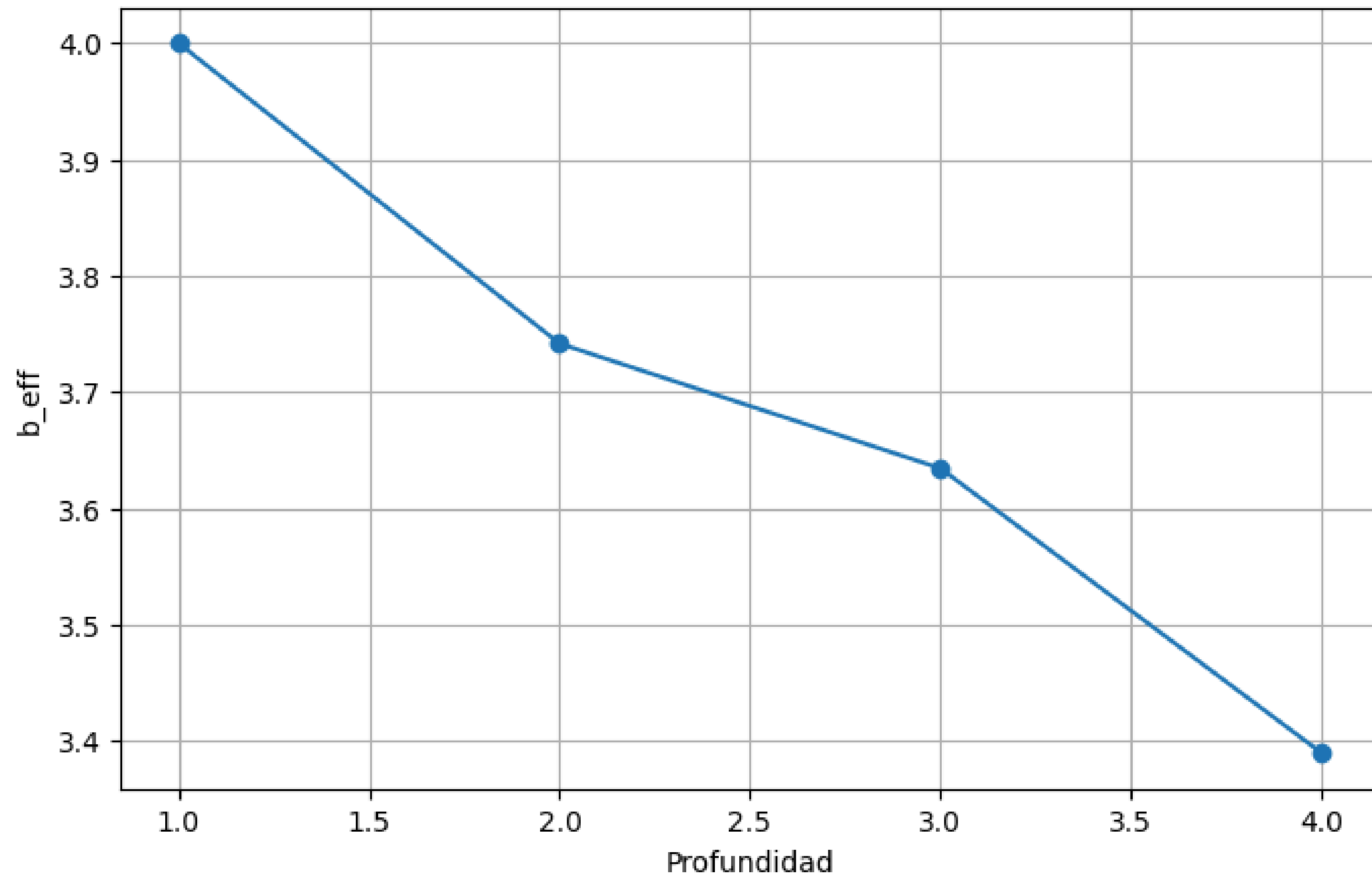
Tiempo promedio búsqueda



Profundidad vs Nodos Explorados



Factor de Ramificación Efectivo





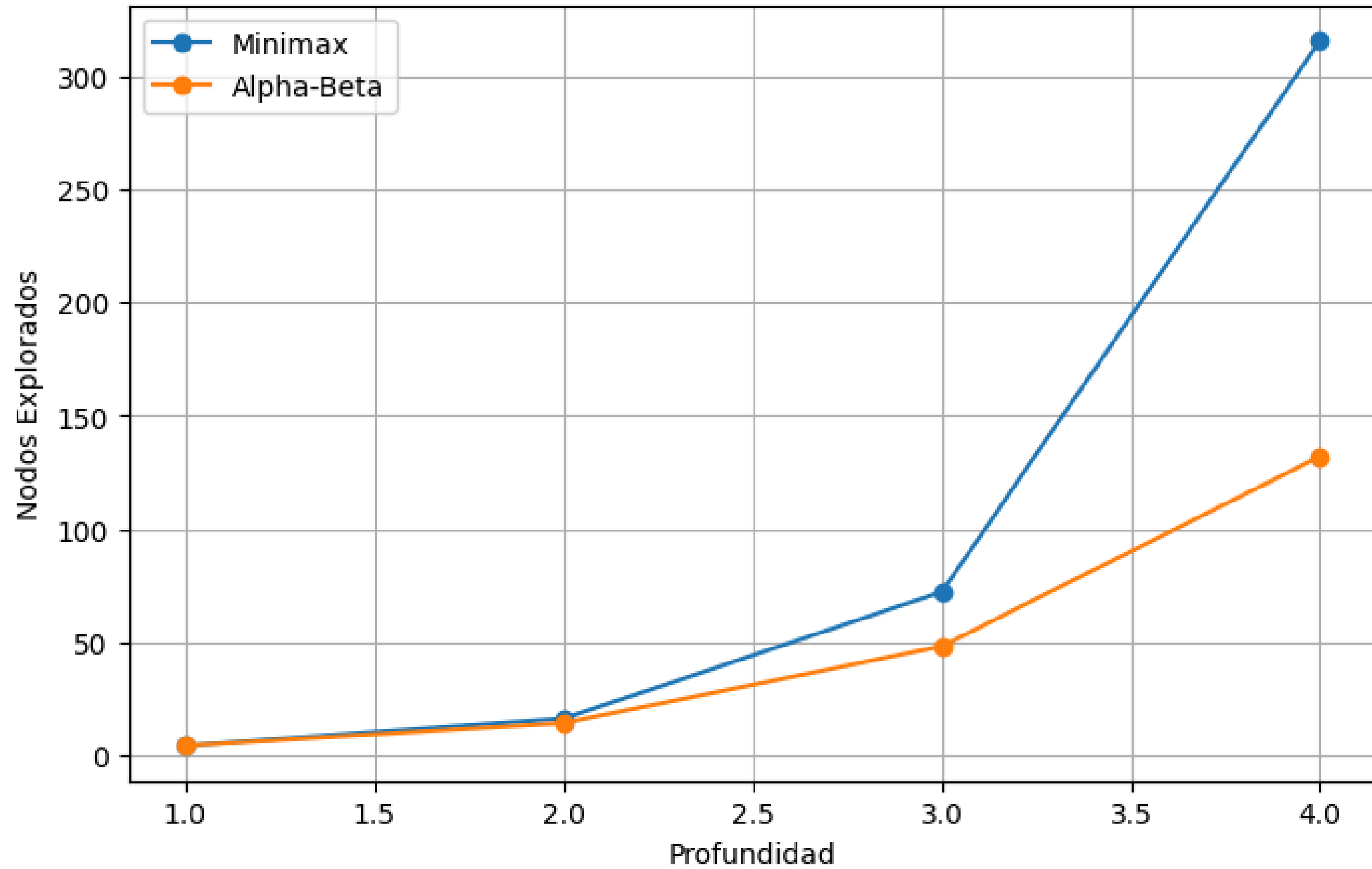
Torneo Alpha-Beta vs Minimax



Se evaluó la cantidad de nodos explorados por ambos algoritmos al incrementar la profundidad de búsqueda de 1 a 4 niveles.

Profundidad	Minimax	Alpha-Beta
1	4	4
2	16	14
3	73	48
4	316	132

Minimax vs Alpha-Beta





DISCUSIÓN



- Alpha-Beta presentó un desempeño superior durante el torneo, obteniendo 17 victorias de 20 partidas gracias a la combinación de búsqueda adversarial y evaluación heurística.
- MCTS logró obtener algunas victorias, pero requirió un tiempo de búsqueda considerablemente mayor debido a la gran cantidad de simulaciones necesarias para estimar la calidad de los movimientos.
- La función heurística basada en movilidad, control de esquinas, control de bordes y diferencia de fichas permitió a Alpha-Beta tomar decisiones más consistentes durante las distintas fases de la partida.
- La comparación entre Minimax y Alpha-Beta evidenció que la poda reduce significativamente la cantidad de nodos explorados, especialmente a mayores profundidades de búsqueda.
- Aunque MCTS mostró una mayor demanda computacional, continúa siendo una alternativa efectiva en problemas donde no se dispone de una función heurística bien definida.
- Los resultados obtenidos demuestran que la poda Alpha-Beta mejora la eficiencia de la búsqueda sin afectar la calidad de la decisión final, permitiendo explorar posiciones más complejas con un menor costo computacional.

: : : : : : : **CONCLUSIONES** : : : : : : :

- Alpha-Beta obtuvo mejores resultados que MCTS durante el torneo, consiguiendo 17 victorias de 20 partidas gracias al uso de poda y una función heurística adaptativa.
- La comparación entre Minimax y Alpha-Beta demostró que la poda reduce significativamente la cantidad de nodos explorados, especialmente a mayores profundidades de búsqueda.
- MCTS presentó un mayor costo computacional debido a la gran cantidad de simulaciones necesarias para evaluar los movimientos.
- La función heurística basada en movilidad, control de esquinas, control de bordes y diferencia de fichas permitió mejorar la calidad de las decisiones tomadas por el agente Alpha-Beta.

⋮⋮⋮⋮⋮⋮⋮ **RECOMENDACIONES** ⋮⋮⋮⋮⋮⋮⋮

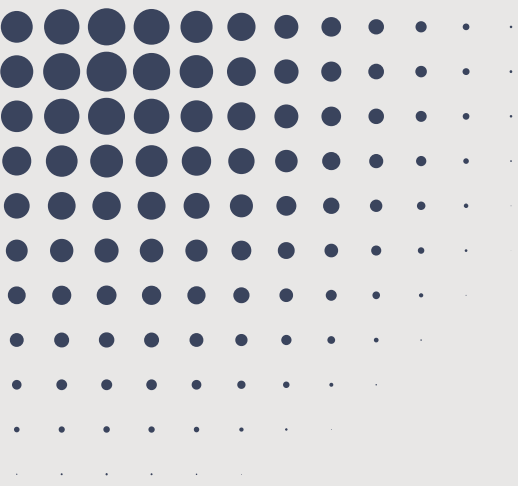
- Incrementar la profundidad de búsqueda de Alpha-Beta para analizar el impacto de una exploración más profunda sobre la calidad de las decisiones y el rendimiento del agente.
- Ajustar parámetros de Monte Carlo Tree Search, como el tiempo de búsqueda y la constante de exploración UCT, para mejorar la calidad de las decisiones obtenidas.
- Considerar versiones híbridas de MCTS que incorporen funciones heurísticas durante las simulaciones para aumentar su competitividad frente a Alpha-Beta.
- Utilizar Alpha-Beta en escenarios donde se requieran decisiones rápidas y consistentes, aprovechando la reducción de estados evaluados proporcionada por la poda.



REFERENCIAS



- NumPy Developers. NumPy Documentation. Extraído de: <https://numpy.org/doc/>
- Pandas Development Team. Pandas Documentation. Extraído de: <https://pandas.pydata.org/docs/>
- Matplotlib Development Team. Matplotlib User Guide. Extraído de: <https://matplotlib.org/stable/users/index.html>
- Pygame Community. Pygame Documentation. Extraído de: <https://www.pygame.org/docs/>
- S. Russell y P. Norvig. Artificial Intelligence: A Modern Approach (4th ed.). Pearson, 2021.
- GeeksforGeeks. Mini-Max Algorithm in Artificial Intelligence. Extraído de: <https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-1-introduction/>
- GeeksforGeeks. Alpha-Beta Pruning in Adversarial Search Algorithms. Extraído de: <https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-4-alpha-beta-pruning/>
- C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis y S. Colton. A Survey of Monte Carlo Tree Search Methods. IEEE Transactions on Computational Intelligence and AI in Games, Vol. 4, No. 1, 2012.
- L. Kocsis y C. Szepesvári. Bandit Based Monte-Carlo Planning. Proceedings of the 17th European Conference on Machine Learning (ECML), 2006.

A large, dark blue trapezoidal shape that serves as a background for the main text. It has a slanted left side and a slanted right side, with a horizontal top and bottom. The text is centered within this shape.

¿DUDAS O COMENTARIOS?

